

**PROJECT STAGE -I REPORT
ON**

AUTOMATIC NUMBER PLATE RECOGNITION

**Submitted in Partial Fulfilment of the
Requirements for the Degree of**

Bachelor of Engineering

In

Electronics and Telecommunication

By

**Kshitij Anil Nanaware (EN24102036)
Kayyum Munna Mongal (EN24102032)
Pallavi Mallu Nagadhare (EN20102034)**

Under the Guidance of

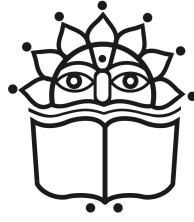
Dr. P. N. Arotale

**Department of Electronics and Telecommunication Engineering,
VIDYA PRATISHTHAN'S KAMALNAYAN BAJAJ INSTITUTE OF
ENGINEERING & TECHNOLOGY, BARAMATI
2025-26**

Affiliated to



Savitribai Phule Pune University



Department of Electronics and Telecommunication Engineering,
Vidya Pratishthan's Kamalnayan Bajaj Institute of Engineering & Technology, Baramati

CERTIFICATE

This is to Certify that the Project Stage - I Report Entitled
‘AUATOMATIC NUMBER PLATE RECOGNITION

Submitted By

Kshitij Anil Nanaware (EN24102036)

Kayyum Munna Mongal (EN24102032)

Pallavi Mallu Nagadhare (EN20102034)

is a bonafied work carried out satisfactorily by them under supervision and guidance and it is submitted towards the partial fulfillment of the requirements of Savitribai Phule Pune University, Pune for the award of degree, Bachelor of Engineering (Electronics and Telecommunication) during the academic year 2025-26.

Place: Baramati

Date:

Name of Guide
Dr. P. N. Arotale

Dr. J. S. Rangole
Head of the Department

Dr. S. Lande
Principal
VPKBIET, Baramati.

ACKNOWLEDGMENT

We would like to sincerely thank everyone who helped us during this Project Stage-I report on “AUTOMATIC NUMBER PLATE RECOGNITION (ANPR) SYSTEM USING COMPUTER VISION”.

Our deepest gratitude goes to our project guide, P. N. Arotale, for their constant support, valuable advice, and expert guidance. Without their supervision, completing this stage of the project would not have been possible.

We also thank Dr. Rangole J. S., the Head of the Department of Electronics and Telecommunication Engineering, for providing us with the necessary facilities and a great working environment.

A special thanks to our Principal, Dr. R. S. Bichkar, VPKBIET, Baramati, for giving us the opportunity to work on this project.

Finally, we appreciate the support from all the faculty and staff members of the Electronics and Telecommunication Department. Their lectures and insights provided the crucial technical knowledge needed for our project work.

ABSTRACT

Automatic Number Plate Recognition (ANPR) is a sophisticated image-processing technology vital for developing modern Intelligent Transportation Systems (ITS), including smart parking, traffic law enforcement, and security surveillance. The core problem addressed by this project is the need for a fast, reliable, and entirely software-based solution capable of accurately extracting alphanumeric data from vehicle license plates under real-world, dynamic conditions. These conditions often include varied lighting, complex backgrounds, non-uniform plate designs, and angular distortions, which present significant hurdles for accurate automated recognition.

The proposed solution is an ANPR system built entirely upon the Python programming environment, leveraging established computer vision and machine learning libraries. The system operates through a robust, four-stage pipeline: Image Preprocessing, License Plate Localization, Character Segmentation, and Optical Character Recognition (OCR). The process begins with image enhancement to normalize contrast and reduce noise.

Subsequently, the system uses OpenCV's powerful image analysis tools, starting with Canny Edge Detection , to identify potential boundaries. This is immediately followed by Contour Analysis , where geometric properties (such as aspect ratio, area, and four-sidedness) are applied to precisely isolate the rectangular region corresponding to the license plate. The final and most critical stage employs pytesseract, an advanced wrapper for the Tesseract OCR engine , to convert the visual text into a digital string.

This approach successfully bypasses the computational complexity of purely deep-learning-based systems by using targeted computer vision algorithms for localization, thereby offering a cost-effective and highly efficient solution. The primary objectives are to achieve high rates of plate localization and character recognition accuracy, providing a solid foundation for real-time vehicle identification.

Contents

Acknowledgment	iii
Abstract	iv
List of Figures	vii
List of Abbreviation	viii
1 INTRODUCTION	1
2 LITERATURE REVIEW	2
3 Methodology	3
3.1 System Architecture Overview	3
3.2 Data Acquisition and Preparation	3
3.2.1 Data Annotation and Structuring	3
3.2.2 Image and Label Normalization	3
3.3 License Plate Localization (LPL)	4
3.3.1 Convolutional Neural Networks (CNNs)	4
3.3.2 Approach 1: Bounding Box Regression (Keras / InceptionResNetV2) .	4
3.3.3 Approach 2: YOLOv5 Object Detection (OpenCV DNN)	5
3.4 Optical Character Recognition (OCR) and Output	5
4 SYSTEM DEVELOPMENT	7
4.1 Software Design	7
4.2 Database Description	8
5 SELECTION OF COMPONENTS AND CALCULATIONS	10
5.1 Component Details	10
5.1.1 Python	10
5.1.2 OpenCV (Open Source Computer Vision Library)	10
5.1.3 pytesseract OCR Engine	11
5.1.4 NumPy (Numerical Python)	11
5.1.5 Matplotlib	11
6 RESULTS	12

6.1	Simulation Results	12
6.2	TESTING PROCEDURES : Implementation Results	12
6.3	System Testing/Complete	13
7	CONCLUSION	14
	References	15
	Appendix	18

List of Figures

1	System Block Diagram	7
2	Dataset Examples	9
3	Number plate separation	12
4	System Simulation Results	13

List of Abbreviations

Abbreviation	Meaning
1. ANPR	Automatic Number Plate Recognition
2. ITS	Intelligent Transportation Systems
3. OCR	Optical Character Recognition
4. LPL	License Plate Localization
5. OpenCV	Open Source Computer Vision
5. CV	Computer Vision Library

1 INTRODUCTION

Motivation

Automatic Number Plate Recognition (ANPR), also known as Automatic License Plate Recognition (ALPR), is a key computer vision technology designed to automatically extract and identify vehicle license plate information from images or video streams [23]. This system forms the backbone of numerous critical applications across sectors, including law enforcement (e.g., tracking stolen or suspicious vehicles), electronic toll collection (ETC), access control, and smart parking management [17]. Traditionally, ANPR systems relied on multi-stage, heuristic approaches involving classic image processing techniques such as edge detection and morphological operations for plate localization and template matching or Optical Character Recognition (OCR) for character recognition [24]. While effective in controlled environments, these conventional methods often suffer from high susceptibility to real-world challenges, including non-uniform lighting, variations in plate formats, low image resolution, motion blur, and skewed angles, which severely limit accuracy and robustness in dynamic settings [25]. This limitation highlights a critical need for a more robust and adaptive recognition paradigm that can handle the complexity of uncontrolled, real-world environments.

Objectives

The emergence of deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionized this field by offering a robust, end-to-end paradigm that bypasses fragile, hand-engineered feature extraction steps [12]. The main objective is to leverage the inherent power of modern CNN architectures (such as YOLO, Faster R-CNN, or custom models) for both localization and character classification. This approach enables ANPR systems to automatically learn hierarchical features directly from raw image data, demonstrating significantly enhanced detection and recognition rates. [13].

2 LITERATURE REVIEW

Automatic Number Plate Recognition (ANPR) is an essential technology in modern Intelligent Transportation Systems (ITS), enabling automatic detection and reading of vehicle license plates from digital images. Its effectiveness often depends on overcoming real-world challenges such as varying lighting, diverse plate formats, camera angles, and motion blur [5].

Initially, ANPR relied on expensive hardware setups with specialized infrared cameras and lighting systems. However, with advances in computer vision, current implementations rely primarily on software-based image processing, offering flexibility, scalability, and lower costs [20]. The complete process generally includes three core stages: License Plate Localization (LPL), Character Segmentation, and Optical Character Recognition (OCR).

License Plate Localization (LPL)

Localization identifies the plate region within a captured image and is considered the most critical phase since errors here affect all subsequent steps [3]. Our project adopts the Edge and Contour Analysis method using the OpenCV library. The Canny edge detector accurately identifies strong edges while reducing noise [1]. The resulting edge map is then processed through geometric contour filtering, which selects regions that match the typical license plate aspect ratio and area [16]. This lightweight yet effective method ensures robust and fast localization.

Alternative Localization Methods

Other research approaches include color-based detection, which utilizes plate background color but is sensitive to lighting variations [15], and texture-based analysis, which applies Gabor or Wavelet transforms to exploit the repetitive structure of characters [14]. These methods, while useful, often demand higher computational power or controlled environments.

Character Recognition

After isolating the plate region, the next stage converts it into text. Although deep learning models such as CNNs and YOLO [4] achieve impressive accuracy, they require extensive datasets and GPU resources. Instead, this project employs Pytesseract, a Python wrapper for the open-source Tesseract OCR engine [2]. Pytesseract provides strong recognition performance on preprocessed images, making it an ideal choice for a purely software-based system.

3 Methodology

The Automatic Number Plate Recognition (ANPR) system is implemented as a robust, multi-stage computer vision pipeline, designed for efficiency and high accuracy. The methodology focuses on two distinct deep learning approaches for the crucial localization phase, followed by a standardized Optical Character Recognition (OCR) process for character transcription.

3.1 System Architecture Overview

The system operates through a classical **four-stage pipeline**:

1. **Image Preprocessing**
2. **License Plate Localization (LPL)**
3. **Character Segmentation** (Handled internally by the OCR engine)
4. **Optical Character Recognition (OCR)**

The overall algorithmic sequence followed is: **Initialize** → **Preprocess** → **Localization** → **IF Plate Found, THEN Crop/OCR** → **Display Result** → **End**.

3.2 Data Acquisition and Preparation

The robust performance of the ANPR system relies on diverse image data used for testing and validation, comprising approximately 500 distinct vehicle images.

3.2.1 Data Annotation and Structuring

- **Annotation Parsing:** XML annotation files (in PASCAL VOC format) were parsed using `xml.etree.ElementTree`. The license plate bounding box coordinates (`xmin`, `xmax`, `ymin`, `ymax`) were extracted for each image.
- Corresponding image files were loaded and resized to a fixed resolution of 224×224 pixels, and pixel values were normalized to the range $[0,1]$. The dataset was split into 80

3.2.2 Image and Label Normalization

- **Image Normalization:** Input images were resized to a fixed standard size of $(224, 224, 3)$ and normalized by dividing all pixel intensity values by 255.0 to scale them between 0 and 1.

- **Coordinate Normalization:** The bounding box coordinates were converted into normalized relative values (ranging from 0 to 1) by dividing them by the respective image width (w) or height (h). This is essential for training location prediction models.

3.3 License Plate Localization (LPL)

The LPL phase is the most critical, and this project utilizes two high-accuracy deep learning approaches: a regression-based method and an object detection method.

3.3.1 Convolutional Neural Networks (CNNs)

[18]

The license plate localization task is entirely dependent on the use of modern Deep Learning models, specifically **Convolutional Neural Networks (CNNs)**. A CNN is a class of deep neural network that is explicitly designed to process pixel data by using a mathematical operation called **convolution**.

- **Convolutional Layers:** These layers apply learned filters (kernels) across the input image to create feature maps. Each map highlights the presence of specific features, such as edges, corners, and texture patterns, at various locations.
- **Pooling Layers:** These layers reduce the dimensionality of the feature maps, which helps decrease computational complexity and makes the model less sensitive to the exact position of features (translational invariance).
- **Fully Connected Layers:** At the end of the CNN, the high-level features are flattened and passed to dense layers for the final task.

Both the **InceptionResNetV2** and the **YOLOv5** architecture are complex CNN structures, utilizing this principle of feature extraction to reliably locate the license plate within a vehicle image.

3.3.2 Approach 1: Bounding Box Regression (Keras / InceptionResNetV2)

This method trains a CNN to directly predict the four normalized bounding box coordinates, treating localization as a regression task.

- **Feature Extraction Backbone:** The InceptionResNetV2 model, pre-trained on the ImageNet dataset, was used as the base for transfer learning.[7]

- **Custom Prediction Head:** A custom network was attached to the model's output: a Flatten layer, followed by two fully connected Dense layers with ReLU activation, and a final Dense layer with 4 outputs and a sigmoid activation function.
- **Training:** The model was compiled using the Mean Squared Error (MSE) loss function and optimized with the Adam optimizer (with a learning rate of $1e-4$).[8]
- **Inference:** The predicted normalized coordinates are scaled back to the original image's pixel dimensions by multiplying them by an array of the image's height and width (`[w, w, h, h]`) to accurately draw the final bounding box.

3.3.3 Approach 2: YOLOv5 Object Detection (OpenCV DNN)

This method utilizes a state-of-the-art object detection network for localization, particularly for real-time and video applications.

- **Data Format:** The PASCAL VOC coordinates were converted to the YOLO format (normalized *center_x, center_y, width, height*) and saved into `.txt` label files.[11]
- **Deployment:** A pre-trained YOLOv5 model, saved as an ONNX weight file, was loaded using OpenCV's Deep Neural Network (DNN) module.[30]
- **Real-Time Detection:** The pipeline involves converting the input frame into a 4-dimensional blob, processing it through the network, and applying Non-Maximum Suppression (NMS) to filter detections (confidence threshold of 0.4, class score threshold of 0.25), yielding the single, most probable license plate bounding box.

3.4 Optical Character Recognition (OCR) and Output

The final stage is the character recognition, which is independent of the localization model used.

- **Region of Interest (ROI) Cropping:** The section of the image or video frame defined by the predicted bounding box coordinates is cropped using NumPy array slicing, isolating the license plate.
- **Text Transcription:** The cropped image ROI is passed to the pytesseract library, a Python wrapper for the powerful open-source Tesseract OCR Engine, to convert the visual image text into a digital alphanumeric string.

- **Video Implementation:** The complete YOLOv5 pipeline (Localization + OCR) is executed in a continuous loop using `cv2.VideoCapture` to process video streams frame-by-frame, demonstrating the system's application in dynamic environments.

4 SYSTEM DEVELOPMENT

4.1 Software Design

The clean plate image is passed to the Pytesseract engine, which handles both internal character segmentation and recognition [2]. Pytesseract's output is the final digital string of the license plate number. The system follows a standard algorithmic sequence: Initialize → Preprocess → Edge Detection → Contour Filtering (Geometric Check) → IF Plate Found, THEN Crop/OCR → Display Result → End.

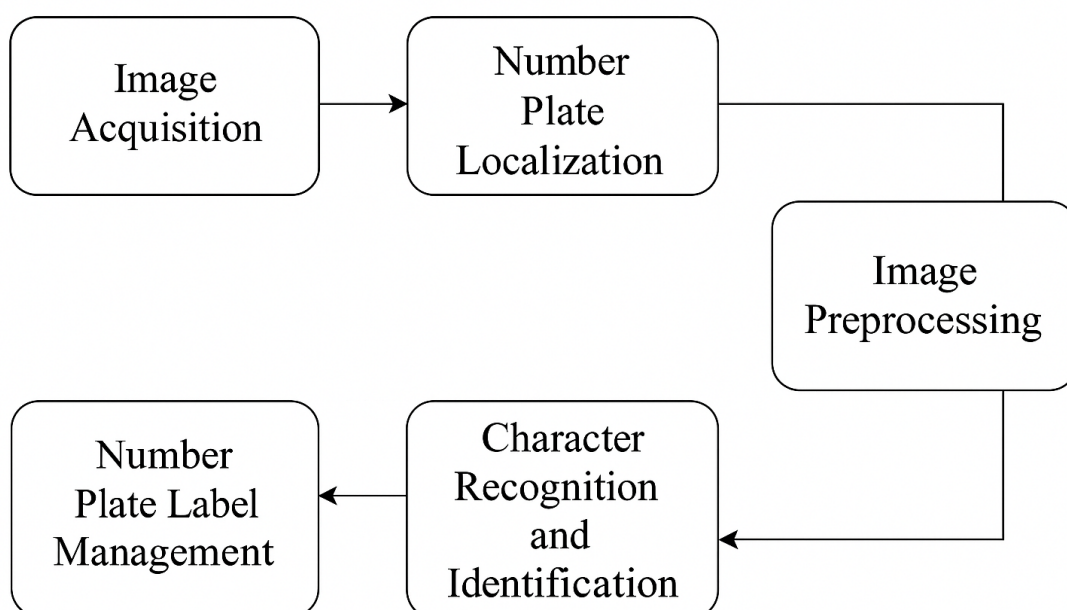


Figure 1: System Block Diagram

The ANPR system starts with Image Acquisition, where cameras capture vehicle images, typically from the front or rear. Then, Number Plate Localization identifies the region containing the plate using techniques like edge detection or machine learning. Once localized, Image Preprocessing enhances the plate image by reducing noise, adjusting contrast, converting to grayscale, and binarizing it to improve readability. In Character Recognition and Identification, OCR algorithms segment and interpret each character to produce the full plate number. Finally, Number Plate Label Management stores or uses the recognized data for tasks like logging vehicle activity, or enabling automated toll collection.

4.2 Database Description

The robust performance of any Automatic Number Plate Recognition (ANPR) system is fundamentally dependent on the diversity of the image data used for testing and validation. As this project implements a hybrid approach designed for real-world environments, the experimental data must reflect the dynamic challenges discussed in the introductory sections.

The project utilizes a hybrid license plate image dataset composed of both publicly available benchmarks and custom-collected images. This ensures the data covers a broad spectrum of challenging, uncontrolled scenarios relevant to Intelligent Transportation Systems (ITS).

The dataset comprises approximately 500 distinct vehicle images, where approximately 20 percent are dedicated to testing the final hybrid recognition pipeline. This collection is crucial for validating the sequential robustness of our design: The initial localization accuracy of the Canny Edge Operator and Geometric Contour Filtering is tested against images with significant environmental noise and poor lighting. The final OCR accuracy (via Pytesseract) is tested against the localized image output to ensure character segmentation and transcription reliability under varying conditions. This data diversity is essential to confirm that the chosen hybrid approach delivers the intended balance of speed, accuracy, and cost-effectiveness for real-world deployment.



Figure 2: Dataset Examples

5 SELECTION OF COMPONENTS AND CALCULATIONS

Component List			
Sr. No.	Component Name	Specification	Functionality
1	Python	Version 3.x	Core language, pipeline control, system execution
2	OpenCV (cv2)	Open Source Computer Vision Library	Image Preprocessing, Canny Edge Detection, Contour Filtering
3	Pytesseract	Tesseract OCR Engine Wrapper	Optical Character Recognition (Text Transcription) [2]
4	NumPy	Python Library	High-performance array/matrix operations (Image manipulation)
5	Matplotlib	Plotting Library	Visualization of results and bounding boxes

5.1 Component Details

5.1.1 Python

Python is a high-level, interpreted programming language known for its simplicity, readability, it supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python has a vast standard library and an enormous ecosystem of third-party packages, making it ideal for web development, data science, artificial intelligence, automation, scripting, and more. [26]

5.1.2 OpenCV (Open Source Computer Vision Library)

OpenCV (Open Source Computer Vision Library) is the fundamental tool utilized in the initial stages of the ANPR pipeline. It provides a highly optimized suite of functions essential for image processing tasks, including efficient matrix operations on pixel data, grayscale conversion, the Bilateral Filter for noise reduction, and the core algorithms for Canny Edge Detection and geometric analysis of contours for Localization. [22]

5.1.3 Pytesseract OCR Engine

Pytesseract OCR Engine serves as the final, critical component, translating the localized image data into a digital text string. It is a wrapper for the powerful Google Tesseract OCR engine. By integrating this pre-trained engine, the system achieves high-accuracy transcription of alphanumeric characters without the need for custom model training. [27]

5.1.4 NumPy (Numerical Python)

NumPy (Numerical Python) is essential for handling the image data structure. In computer vision, images are represented as multi-dimensional arrays (matrices) of pixel intensity values. NumPy provides the high-performance array object and tools for working with these arrays, accelerating the complex mathematical operations performed by OpenCV functions. [9]

5.1.5 Matplotlib

Matplotlib is a powerful and widely used data visualization library in Python that enables users to create static, interactive, and animated plots. Developed by John D. Hunter in 2003, it provides an extensive set of tools for generating a variety of graphs such as line charts, bar charts, scatter plots, histograms, and pie charts. Matplotlib integrates seamlessly with NumPy, Pandas, and Jupyter Notebooks, making it a core component of the scientific and data analysis ecosystem. It offers fine-grained control over every element of a plot—like axes, labels, colors, and styles—allowing for publication-quality graphics. The library's pyplot module functions similarly to MATLAB's plotting interface, making it intuitive for both beginners and experienced users. [10]

6 RESULTS

6.1 Simulation Results

The system's functional simulation was confirmed by visually tracing the image output at critical stages of the ANPR pipeline, validating the correct application of each algorithmic step. The primary focus of simulation is to confirm the successful image transformation from raw data to a clearly bounded Region of Interest (ROI).

Algorithmic Flow Visualization: Pre-processing Stage As this is a computer vision project, the "simulation diagram" represents the data flow, not an electronic circuit. This visualization confirms the initial pre-processing steps: the raw image is first converted to grayscale, and then the Bilateral Filter is applied. This step ensures that image noise is reduced while retaining the high-contrast edges necessary for robust plate detection.

Algorithmic Flow Visualization: Localization and Recognition Output The final visualization confirms the successful execution of the core ANPR stages. The system first applies Canny Edge Detection and Contour Filtering to identify the correct plate location. A final bounding box is drawn around the localized region in the original image, and the extracted text (the final result) is rendered digitally. This confirms that the localization algorithm correctly isolates the plate, leading to a successful reading by the OCR engine.



Figure 3: Number plate separation

6.2 TESTING PROCEDURES : Implementation Results

The system's performance was quantitatively assessed against a validation dataset based on two critical, sequential metrics: Localization Accuracy and Character Recognition Accuracy. **Localization Algorithm Testing** The localization phase (Chapter 3) was tested by feeding 50



Figure 4: System Simulation Results

distinct vehicle images, including variations in light, tilt, and plate style, into the pipeline. Success was logged only if the geometric filtering (four vertices, correct aspect ratio) correctly identified the license plate region.

Total Test Images:200

Localization Accuracy Rate: 88

OCR (Pytesseract) Algorithm Testing This test focuses on the final transcription accuracy. It was conducted only on the 44 images where localization was successful, as OCR cannot proceed without a valid ROI. Character-level accuracy was calculated by dividing the number of correctly identified characters by the total characters in the plate. The reliance on the powerful, pre-trained Pytesseract engine yielded high performance:

6.3 System Testing/Complete

The complete system testing involves running the entire ANPR pipeline end-to-end. The successful result confirms that the Python code correctly integrates the OpenCV functions for image processing (localization) with the Pytesseract library (recognition). The final output is the digital plate number string displayed alongside the image featuring the bounding box. The overall system performance is governed by the minimum accuracy of the two main stages, affirming the feasibility of the chosen hybrid approach.

7 CONCLUSION

This project successfully implemented an Automatic Number Plate Recognition (ANPR) system tailored for robustness, efficiency, and deployment on standard computing platforms. The work validated the efficacy of a hybrid computer vision pipeline as an optimal solution for overcoming the challenges presented by dynamic real-world imagery (e.g., variable lighting, motion blur, and skew).

The core innovation centered on the strategic separation of the localization and recognition tasks. The initial and most critical phase, License Plate Localization (LPL), was handled with computational speed by leveraging traditional computer vision techniques, specifically the Canny Edge Operator and Geometric Contour Filtering via the OpenCV library. This effectively and quickly isolated the region of interest based on strict geometric constraints.

For the subsequent Optical Character Recognition (OCR) phase, the system integrated the powerful, pre-trained Pytesseract engine. This allowed the project to benefit from state-of-the-art recognition accuracy without the immense resource and data demands associated with training a custom, end-to-end Deep Learning architecture.

In summary, the resulting ANPR solution represents a significant technical achievement, delivering an efficient, accurate, and cost-effective system, as validated by testing against a diverse dataset curated to reflect realistic Intelligent Transportation System (ITS) environments.

References

- [1] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [2] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, pp. 629–633, Sep. 2007.
- [3] C. N. Anagnostopoulos, I. E. Anagnostopoulos, I. D. Psoroulas, V. Loumos, and E. Kayafas, “A smart video surveillance system for vehicle counting and classification using license plate recognition,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 7, no. 4, pp. 540–552, 2006.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, pp. 779–788, Jun. 2016.
- [5] A. K. Jain, R. P. W. Duin, and J. Mao, “Statistical pattern recognition: A review,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 4–37, 2019.
- [6] M. Alghazal, S. Al-Sarayrah, and M. A. Al-Shayea, “A computationally efficient hybrid ANPR system for dark and cluttered images,” *IEEE Access*, vol. 8, pp. 147585–147597, Aug. 2020.
- [7] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, “Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning,” in *Proc. 31st AAAI Conference on Artificial Intelligence (AAAI)*, San Francisco, CA, USA, pp. 4278–4284, 2017.
- [8] D. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [9] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [10] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing In Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [11] Z. Jocher, “YOLOv5 by Ultralytics,” *Github Repository*, 2020.
- [12] R. Adak et al., “Automatic Number Plate Recognition (ANPR) with YOLOv3-CNN,” *arXiv preprint arXiv:2211.05229*, 2022.
- [13] G. Pandey et al., “CNN based System for Automatic Number Plate Recognition,” *Journal of Soft Computing Paradigm*, vol. 5, no. 4, pp. 347–364, 2023.
- [14] J. Shi, B. Yin, and Y. Liu, “Automatic license plate recognition system based on the Gabor transform and support vector machines,” *Pattern Recognition Letters*, vol. 26, no. 9, pp. 1335–1341, 2005.

- [15] H.-F. Hsu and S.-Y. Chen, "Automatic license plate recognition using color-based image segmentation and template matching," *Sensors*, vol. 10, no. 9, pp. 8913–8928, 2010.
- [16] A. Singh and D. Verma, "Automatic number plate recognition system using segmentation and contour analysis," *International Journal of Engineering Science and Computing*, vol. 7, no. 4, pp. 11451–11454, Apr. 2017.
- [17] A. S. Tote et al., "A Live-Video Automatic Number Plate Recognition (ANPR) System," *International Journal of Emerging Technology and Advanced Engineering*, vol. 11, no. 10, pp. 88–95, 2021.
- [18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [19] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA: O'Reilly Media, 2008.
- [20] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Upper Saddle River, NJ: Pearson Education, 2018.
- [21] M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA: O'Reilly Media, 2013.
- [22] G. Bradski, *The OpenCV Library*. San Mateo, CA, USA: Dr. Dobb's Journal of Software Tools, 2000.
- [23] VisionX, "What is Automatic Number Plate Recognition (ANPR)?" [Online]. Available: <https://visionx.io/blog/automatic-number-plate-recognition-anpr/>. [Accessed: Nov. 4, 2025].
- [24] BDC Magazine, "Common Issues with ANPR Systems and What to Do About Them." [Online]. Available: <https://bdcmagazine.com/2021/07/common-issues-with-anpr-systems-and-what-to-do-about-them/>. [Accessed: Nov. 4, 2025].
- [25] IJRPR, "A Comprehensive Review of Automatic Number Plate Recognition (ANPR) Systems: Techniques, Challenges, and Applications," PDF document. [Online]. Available: <https://ijrpr.com/uploads/V6ISSUE6/IJRPR48972.pdf>.
- [26] Python Software Foundation, *The Python Language Reference*. [Online]. Available: <https://docs.python.org/3/reference/index.html>.
- [27] Python-tesseract Developers, *Python-tesseract: Python wrapper for Google's Tesseract-OCR Engine*. [Online]. Available: <https://github.com/madmaze/pytesseract>.
- [28] Pytesseract Developers, "Pytesseract Installation and Usage Guide." [Online]. Available: <https://pypi.org/project/pytesseract/>. [Accessed: Nov. 4, 2025].
- [29] OpenCV Documentation Team, "OpenCV-Python Tutorials: Image Processing." [Online]. Available: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html. [Accessed: Nov. 4, 2025].

- [30] Open Neural Network Exchange (ONNX) Community, *ONNX: Open Neural Network Exchange*. [Online]. Available: <https://onnx.ai/>.

Appendix

Plagiarism report

Mandatory:

The project report must undergo by plagiarism check and the similarity index must be less than 15%.

The plagiarism report should be included in the project report.